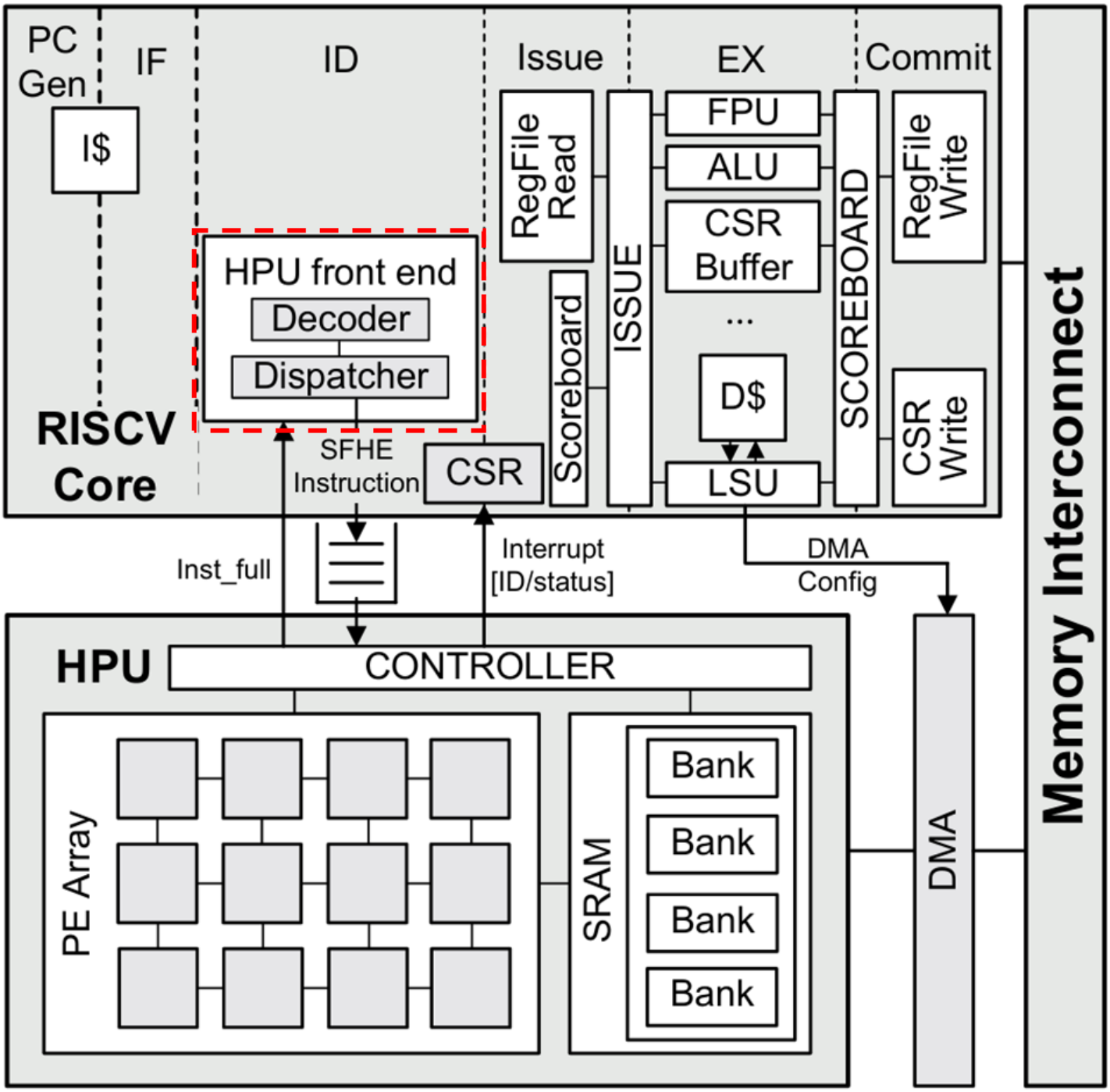
HPU

硬件架构 4.7更新

1 整体设计



本版设计不再假设 HPU 内部存在一组大容量独立 VRF，而是以 **统一的本地 SRAM** 作为主要数据驻留层，在 PE 阵列前后仅保留小型流式缓冲，用于吸收访存延迟并支撑流式执行。与此同时，NTT/INTT 与独立 Shuffle 指令在物理实现上复用同一条 **128 并行重排/蝶形执行通路**，其中 NTT 阶段内部所需的

重排模式由硬件自动给出，而对软件显式可见的 `pshuf` 仍保留独立扩展能力，以支持自同构及后续其他重排类操作。该组织方式延续了原文档中“custom0 负责 HPU 内部语义、custom1 负责外部 DMA 配置”的总体边界，但在内部数据组织与执行粒度上进行了面向资源约束的重构。

1.1 RISC-V核内的HPU接口

RISC-V 核在译码阶段识别自定义操作码 `custom0` 和 `custom1`。核内对 HPU 的支持仍保持“**最小识别、最小派发**”原则：核仅判断指令属于哪一类自定义路径，而不在核内展开 HPU 内部语义。

对于 `custom0` 指令，核侧只负责将 HPU 内部命令送入 HPU 的命令队列。由于 `custom0` 的 opcode 固定，当前设计中更关注的并非完整 32 bit 指令，而是其内部可用编码字段，因此核到 HPU 的主命令接口优先采用 `payload` 形式，传递 HPU 真正关心的内部编码信息。该命令进入 HPU 指令队列后，由 HPU 自身完成后续译码、资源检查与执行调度。这样可以继续保持核与 HPU 的解耦，避免将 HPU 内部执行细节引入核内的常规执行路径。

对于 `custom1` 指令，语义边界与 `custom0` 明确区分。`custom1` 主要服务于 **主存与 HPU 本地 SRAM 之间的数据搬运配置**：CPU / 核侧负责给 DMA 配置主存地址、传输长度、方向及启动信息，而 HPU 侧并不直接感知完整的主存地址与长度参数。与之对应，HPU 只在 SRAM 侧参与 block 级协调，例如某一数据块是否已被 DMA 占用、何时释放、何时完成等。因此，`custom1` 实际上形成两条后端路径：一条通向 DMA 配置面，一条通向 HPU 内部 SRAM 管理器的占用/完成同步逻辑，而不会进入 `custom0` 那条计算命令 FIFO。

在这一接口划分下，RISC-V 核与 HPU 之间仍保持异步并行关系：核在成功派发 `custom0` 或完成 `custom1` 的 DMA 配置后即可继续执行后续指令，而 HPU 在本地独立完成计算、重排、采样和数据搬运过程。这样的分工方式与原版的总体接口边界保持一致，但进一步强化了“核只识别路径，不承担 HPU 内部状态管理”的设计取向。

1.2 HPU控制器与执行单元

HPU 控制器采用“**顺序发射骨架 + 局部状态机执行**”的组织方式。其前端由命令队列、解码器、轻量级 scoreboard 和发射控制构成。命令从 `cmd_queue` 中按序取出后，先由解码器识别指令类别，再由 scoreboard 检查必要的 RAW/WAW、结构冲突、配置上下文冲突及 SRAM 资源占用情况；满足条件后，控制器将该指令分派到对应执行路径。首版设计不追求复杂乱序和多发射能力，而以“安全发射、可验证、可实现”为优先目标。

在数据组织方面，本版 HPU 取消了“大容量独立 VRF + 内部 SRAM”双层数据驻留结构，而改为以 **统一本地 SRAM** 作为主要数据层。也就是说，大块多项式数据、中间结果、DMA 搬入的数据以及待处理对象，均驻留在 HPU 内部的 banked SRAM 中。为保证 PE 阵列能够流式工作，在执行单元前后仅设置小型 `operand/result buffer`，它们的职责不是长期保存完整向量，而是在执行过程中完成按 chunk 取数、送数和结果暂存。这样既避免了独立大 VRF 带来的容量开销，也更符合当前资源受限下“数据以本地对象形式驻留、计算按流式窗口推进”的总体思路。

控制器内部还维护一组较小的状态/上下文寄存器，用于保存当前活动模参数、随机数生成状态、以及独立 Shuffle 所需的模式信息等。与早期方案相比，这里的状态组织更偏向“当前活动上下文”，而不是维护大量长期并存的复杂状态槽位。为解决 25 bit 内部编码空间有限而又需要指向本地数据对象的

问题，控制器可维护少量本地对象描述信息，用于标识统一本地 SRAM 中待计算或待搬运的数据区间；这些描述信息是对本地数据对象的命名，而不是另一层独立数据存储。

在执行结构上，本版 HPU 不再将主要计算资源理解为多套彼此独立的大型专用后端，而是将其组织为一组 **可重构的统一 PE 阵列**。控制器根据指令类型、当前上下文以及数据通路需求，对该阵列施加不同配置，使其在不同计算任务下表现出不同的执行模式和物理并行度。

除统一 PE 阵列外，HPU 还包含与其配套的随机/采样执行路径，以及 SRAM 管理与流式访存控制路径。随机/采样路径面向 `pseed`、`psample` 等操作，由控制器通过当前活动 seed/state 驱动 PRNG 单元按流式方式产生随机数序列，并将结果写回本地对象对应的数据区域。SRAM 管理与流式访存控制路径则由 `sram_mgr` 和相关 load/store 控制逻辑组成，负责统一管理本地 SRAM 的 block 状态、HPU 与 DMA 的 ownership 关系，以及计算数据从本地 SRAM 到执行缓冲区、再从结果缓冲区写回本地 SRAM 的流式搬运过程。

因此，当前 HPU 的执行结构应理解为：**以一组可重构统一 PE 阵列为核心计算资源，在不同指令语义下切换不同执行模式；同时配合小型流式缓冲、随机/采样路径以及统一 SRAM 管理路径，共同构成完整的 HPU 控制与执行体系。**

1.3 中断驱动的同步机制

HPU 与 RISC-V 核之间的同步仍通过专用中断线实现，但在本版设计中，同步语义不再主要依附于某一类具体计算或存储指令，而是通过一条独立的 **`psync` 同步指令** 来显式建立软件可见的阶段边界。其核心思想是：RISC-V 核负责异步派发 HPU 指令流，HPU 在内部顺序发射并独立执行；当软件需要确认某一批 HPU 内部操作已经全部完成并对后续阶段可见时，可在 `custom0` 指令流中插入 `psync`，由 HPU 在满足同步条件后主动向核侧发出中断通知。这样，计算语义与同步语义得以解耦，核侧无需轮询 HPU 状态，也不必将中断机制绑定到某一条特定算术或内部访存指令上。

在这一机制下，RISC-V 核与 HPU 的异步并行关系得以保留。核侧在派发一批 `custom0` 指令及其后的 `psync` 后即可继续执行后续程序；HPU 则在内部独立完成此前命令的执行与收敛。当 `psync` 对应的同步条件成立后，HPU 再通过中断通知核侧进入软件同步点。这种方式使 `psync` 成为 HPU 命令流中的显式阶段屏障：它既不会改变前序指令的执行语义，也不会要求核侧在派发后立即阻塞等待，而是通过“前序收敛—中断通知—软件继续”的方式，建立起适合当前控制架构的同步模型。

因此，在本版整体设计中，中断机制的主要作用可概括为：**不是对单条计算指令完成的被动响应，而是对 `psync` 所定义的阶段性完成条件的显式通知。** 这一调整使中断驱动同步机制从“依附于特定操作”转变为“服务于命令流阶段边界”，更符合 HPU 作为独立 IP、采用顺序发射与局部状态机执行的控制组织方式。

1.4 关键数据通路

结合上述模块划分，本版 HPU 的关键数据通路主要包括以下几条：

1. **指令派发通路**：RISC-V 核的 Decoder / Dispatcher 将 `custom0` 命令送入 `cmd_queue`，随后由 HPU 内部完成解码、scoreboard 检查和顺序发射。这一路径保证核侧只承担最小派发责任，而 HPU 内部完整拥有指令语义与资源调度控制权。

2. **DMA 配置与 SRAM 协调通路**：`custom1` 在核侧配置 DMA 后，一方面驱动主存与 HPU 本地 SRAM 之间的数据传输，另一方面通过 block 级同步接口与 `sram_mgr` 协调对应 SRAM 区块的占用、释放与完成状态。这样可以保证 HPU 在不掌握主存地址细节的前提下，仍能正确感知本地数据对象何时可被计算或回写。
3. **本地数据流式执行通路**：本地对象数据驻留在统一 SRAM 中。执行时，流式 load/store 控制逻辑按当前对象描述和操作类型，从本地 SRAM 中连续读取数据到 `operand buffer`；随后数据进入基础向量算术后端或 NTT/Shuffle 复用后端完成计算；结果再经 `result buffer` 写回统一 SRAM。对基础算术而言，流式粒度按 64 元素组织；对 NTT / Shuffle 而言，流式粒度按 128 并行重排/蝶形路径组织。这是本版 HPU 的核心计算数据通路。
4. **状态/上下文控制通路**：模参数、随机种子、Shuffle 模式及少量对象描述信息由控制器内部状态寄存器维护，并在指令执行时提供给各后端。这里的状态并非另一层数据存储，而是用于约束当前运算模式、数据解释方式和执行后端行为的控制上下文。
5. **中断反馈通路**：当 `custom0` 指令流中的 `psync` 到达队首后，HPU 控制器停止继续发射其后的 younger 指令，并等待此前所有已发射的内部操作全部完成且结果对统一 SRAM 可见。待同步条件满足后，HPU 控制器通过中断线将“阶段完成”信息反馈至核侧中断控制逻辑，随后由软件继续发起下一阶段计算、配置 DMA，或执行后续处理。这一路径使核侧无需主动轮询 HPU 内部状态，即可在显式同步点上获得 HPU 执行收敛的通知。

综上，本版 HPU 的总体设计可以概括为：**核侧最小接口、HPU 内部顺序发射控制、统一本地 SRAM 数据层、同一 PE 阵列在不同模式下工作，以及基于中断的异步同步机制。** 这一结构既继承了原版中 custom0/custom1 分流、异步执行与真实 Shuffle 的设计思想，又针对当前片上资源约束对内部存储组织和执行粒度进行了收缩与重构。

2 指令编码和流水线行为

按照核侧流水线行为划分，HPU 相关指令主要分为两类：一类是通过 `custom0` 进入 HPU 内部命令队列的内部指令，包括基础模运算、NTT/INTT stage、Shuffle、随机/采样、模上下文装载以及同步指令；另一类是通过 `custom1` 配置外部访存的指令，包括 `DLoad` 和 `DStore`，用于主存与 HPU 本地 SRAM 之间的数据搬运。

本章仅从 **RISC-V 核侧行为** 出发描述核内需要支持的译码、派发和字段含义；至于 HPU 内部如何维护对象槽位、状态寄存器和执行路径，则在后续章节展开。

2.1 HPU 内部指令（custom0）

HPU 内部指令统一采用 RISC-V `custom0` 操作码。由于 `custom0` 的 `opcode` 固定，核侧在译码阶段只需识别该指令属于 HPU 内部命令路径，而不解析其具体语义。随后在 dispatch 阶段将该指令送入 HPU 的命令队列，由 HPU 内部自行完成译码、资源检查和执行调度。

31	7	6	0
payload		0001011	

在核侧流水线中，`custom0` 指令的行为如下：

1. **decode 级识别**
   译码器识别出 `custom0` 操作码，并将其标记为 HPU 内部命令。
2. **dispatch 级派发**
   若 HPU 命令队列可写，则将指令派发给 HPU；若队列已满，则 dispatch 级停顿，直到 HPU 队列出现空闲位置。
3. **retire / complete 级完成**
   一旦指令成功派发到 HPU，核侧即认为该指令完成，不等待其在 HPU 内部真实执行结束，也不向通用寄存器写回结果。

因此，从核侧视角看，`custom0` 指令的关键在于“**是否成功进入 HPU 队列**”，而不是“**HPU 是否已经完成该运算**”。

2.2 HPU 外部访存指令（custom1：DLoad / DStore）

HPU 外部访存指令采用 RISC-V `custom1` 操作码，用于主存与 HPU 本地 SRAM 之间的数据传输。与早期仅把 `custom1` 理解为 DMA 配置不同，本版中 `custom1` 路径上的两条指令 `DLoad` 和 `DStore` 同时承担**对象槽位建立/更新**的语义：
`DLoad` 在把数据搬入 HPU 本地 SRAM 的同时，建立某个活动对象槽位；`DStore` 在把对象数据导出到主存的同时，完成对该对象槽位的状态更新或释放。

核侧只需要支持以下字段解释：

- `rs1`：存放主存基址的通用寄存器编号
- `rs2`：存放对象逻辑长度 `l` 的通用寄存器编号
- `rd[2:0]`：对象槽位编号 `obj_id`
- `funct3[0]`：`DIR` 方向位
  - `0`：`DLoad`
  - `1`：`DStore`

当 `DIR = 0`（即 `DLoad`）时，`funct3[2:1]` 解释为 `LOAD_TYPE`：

- `00`：普通片段对象
- `01`：完整多项式对象
- `10`：模上下文对象
- `11`：Shuffle 配置对象

当 `DIR = 1`（即 `DStore`）时：

- `funct3[2]`：`REL` 释放位
  - `0`：导出完成后对象槽位保留

- `1`：导出完成后对象槽位释放
- `funct3[1]`：保留

也就是说，核侧只需在 execute 级读取：

- `rs1` 对应的主存地址
- `rs2` 对应的逻辑长度 `l`

并将这些值连同 `obj_id`、`FULL/REL` 和 `DLoad/DStore` 类型一起送入 `custom1` 配置路径。
HPU **不直接访问 CPU 的通用寄存器**，因此核内不需要新增“HPU 读取 GPR”的特殊通路；所需参数仍由核侧在 execute 级读取后，通过已有的自定义指令配置路径送出。

在流水线行为上，`custom1` 指令仍然是“**配置后即完成**”的模式：

1. decode 级识别 `custom1`，并将指令送入 HPU
2. execute 级读取 `rs1/rs2` 的值并形成外部访存配置
3. 将配置送入 DMA 相关接口
4. 指令在核侧完成，不等待真实的数据搬运结束

因此，从核侧视角看，`DLoad/DStore` 的完成语义是“**外部访存已成功发起**”，而不是“**数据搬运已完成**”。

2.3 基于 `psync` 的同步行为

在原始方案中，中断同步主要依附于带中断使能位的 `sstore` 之类内部访存语义；而在本版中，同步行为通过专门的 `psync` 指令来建立，因此第 2 章中的同步描述也应相应更新。

其流程可概括如下：

1. **`psync` 指令派发**
   当 RISC-V 核执行到一条 `psync` 指令时，该指令作为普通 `custom0` 命令被前端派发至 HPU 命令队列。核侧不等待其实际完成，而是继续执行后续指令，除非 HPU 命令队列满而导致 dispatch 停顿。
2. **HPU 识别同步屏障**
   当 `psync` 到达 HPU 命令队列队首时，控制器将其识别为同步屏障。此时，HPU 不再发射 `psync` 之后的 younger `custom0` 指令，而是等待 `psync` 之前所有已发射的内部操作收敛完成。
3. **前序执行收敛**
   `psync` 的等待范围包括此前所有已发射的 `custom0` 内部操作，如基础模运算、NTT / INTT stage、显式 Shuffle、随机采样及相关结果写回过程。只有当这些操作全部完成，且其结果已经对 HPU 内部统一 SRAM 可见时，控制器才认为同步条件满足。

4. **中断触发**
   当前序指令全部完成后，HPU 控制器将 `psync` 标记为完成，并通过专用中断线向 RISC-V 核发送同步完成中断。该中断所表示的语义是：“此前一批 HPU 内部 `custom0` 操作已经全部完成，可进入下一阶段”。
5. **核内中断响应与后续处理**
   RISC-V 核在中断检测阶段识别到中断有效后，按照常规中断流程保存现场并跳转到中断服务程序；随后软件可根据当前阶段需要继续发起下一批 HPU 指令、配置 DMA，或执行后续处理逻辑。这样，中断不再是对某条具体内部访存指令的附属反馈，而是对 `psync` 所定义的阶段性同步条件的显式通知。

需要特别指出的是，`psync` 的同步边界仅覆盖 **HPU 内部 `custom0` 执行域**，而不默认等待 `custom1` 所配置 DMA 传输的完成。DMA 完成仍应通过 DMA 自身的完成反馈和 `sram_mgr` 的 block 状态更新来体现。这样可以保证内部执行同步与外部数据搬运同步之间的职责分离。

3. HPU 指令助记符设计

为支持 HPU 面向多项式模运算、NTT/INTT、随机采样、真实数据重排以及阶段同步等核心流程，本文定义一组 HPU 指令助记符。它们分为两类：

- **内部执行类指令**：经 `custom0` 进入 HPU 命令队列，由 HPU 内部执行；
- **外部访存类指令**：经 `custom1` 发起主存与 HPU 本地 SRAM 之间的数据搬运，并建立或更新活动对象槽位。

与早期方案相比，本版不再以“大容量向量寄存器文件 + 内部 SRAM”的双层数据组织为前提，而是以 **统一 SRAM + 少量活动对象槽位** 为核心数据组织方式。

3.1 设计原则

本版指令集遵循以下原则：

1. **面向本地数据对象组织**

在当前资源约束下，HPU 内部不再适合继续采用“大容量独立 VRF + 内部 SRAM”的双层数据层次，而改为以 **统一本地 SRAM** 作为主要数据驻留层。计算、重排、采样等操作均围绕本地数据对象展开，PE 阵列前后仅保留小型流式缓冲，用于支持数据按片段连续送入和结果回写。因此，本版指令中的源和目的不再优先理解为“大容量向量寄存器”，而应理解为对本地 SRAM 中当前活动对象或对象片段的引用。

2. **对象描述符表示活动对象/片段视图**

本版中的 `pdst`、`psrc1`、`psrc2`、`psrc` 等字段并不表示传统意义上的向量寄存器号，而表示 **活动对象/片段视图描述符**。
这些描述符由 HPU 控制状态维护，可指向：

- 一个完整多项式对象；
- 一个对象的局部片段；

- 一个对象在当前执行模式下的逻辑视图。

其中，`pntt / pintt` 等需要全局数据依赖的指令，应作用于完整多项式对象；而 `padd / psub / pmul / pmac / pshuf / psample` 等指令，则可以作用于片段对象，以适应资源受限条件下的流式访存和局部计算。

3. **Shuffle 为真实重排操作**

Shuffle 仍然是本版指令集中的核心能力之一。其原因与原文档保持一致：NTT、自同构以及其他提取/重排类操作均要求在 HPU 内部对数据顺序进行真实调整，而不是仅在地址解释层做逻辑映射。因此，本版仍保留独立的 Shuffle 指令能力。不同之处在于：对于 NTT/INTT 执行过程中所需的部分重排，硬件可以在内部复用同一条 Shuffle 数据通路而不要求软件逐次显式配置；但对于自同构和后续可扩展的重排类操作，`pshuf` 仍然作为软件可见的独立指令保留。

4. **NTT / INTT 采用 stage 级指令粒度**

本版中，`pntt` 和 `pintt` 表示单个 stage 的蝶形执行。这样做的原因在于：一次完整变换需要经历多个 stage，每个 stage 内又包含 butterfly 计算、twiddle 推进以及必要重排；若将整个过程压缩为单条指令，则会使单条指令语义过于庞大，不利于控制器和软件之间建立清晰边界。因此，本版采用“**软件显式推进 stage，硬件在 stage 内部自动完成细节**”的语义划分：软件负责发起当前 stage，硬件负责在该 stage 内推进 twiddle、完成 butterfly 计算并调用必要的内部重排。

5. **同步语义从具体操作中解耦**

原方案中的中断同步主要依附于某些具体指令；而本版中，同步能力通过独立的 `psync` 指令显式提供。这样，计算、重排、采样等功能指令与阶段同步边界被分离：软件可在一批 HPU 指令之后插入 `psync`，要求 HPU 在前序操作全部完成后向核侧发出同步完成中断。这使同步语义不再绑定于某一类具体运算，而成为 HPU 命令流中的显式屏障能力。

常量、小控制状态与大数据对象分离处理

本版将“**大数据对象**”和“**小控制状态**”分开处理：

- 大数据对象通过 `DLoad / DStore` 建立为对象槽位；
- 小常量优先通过指令控制字段直接编码；
- 模上下文通过 `pmodld` 从对象槽位装载；
- `pseed` 直接采用立即数种子形式；
- Shuffle 配置通过 `pshcfg` 从对象槽位装载。

3.2 操作数记号约定

为统一表述，本文对本版指令中的记号作如下约定：

- `pdst`：目的活动对象/片段视图描述符
- `psrc1`、`psrc2`：源活动对象/片段视图描述符
- `psrc`：单源对象/片段视图描述符

- `pmod`：模上下文装载相关参数
- `pshf`：Shuffle 模式或模板编号
- `pseed`：随机数种子或 PRNG 初始配置
- `pstage`：NTT / INTT 当前 stage 标识
- `pmode`：执行模式、路径选择或控制位
- `l`：对象的逻辑长度

需要强调的是，`pdst / psrc1 / psrc2 / psrc` 对应的是 HPU 内部当前活动对象或片段视图，而不是另一层独立的大容量寄存器文件。
其中：

- 对 `pntt / pintt`，这些描述符应对应完整多项式对象；
- 对 `padd / psub / pmul / pmac / pshuf / psample`，这些描述符可以对应对象片段。

3.3 指令助记符与功能说明

类别	助记符	操作对象	功能说明	执行模式 / 物理并行度	补充说明
外部访存	dload	mem -> obj	将主存中的数据搬入 HPU 本地 SRAM，并建立/更新对象槽位	custom1 路径 / DMA	LOAD_TYPE=00/01/10/11 分别表示普通片段对象、完整多项式对象、模上下文对象和 Shuffle 配置对象
外部访存	dstore	obj -> mem	将对象槽位对应的数据导出到主存，并更新/释放对象槽位	custom1 路径 / DMA	REL=1 时导出完成后释放对象槽位
基础模运算	padd	psrc1, psrc2 -> pdst	对两个源对象或片段执行逐元素模加，结果写入目的对象	统一 PE 阵列，64 并行逐元素模式	依赖当前活动模上下文
基础模运算	psub	psrc1, psrc2 -> pdst	对两个源对象或片段执行逐元素模减，结果写入目的对象	统一 PE 阵列，64 并行逐元素模式	依赖当前活动模上下文
基础模运算	pmul	psrc1, psrc2/imm -> pdst	对对象执行逐元素模乘，第二源既可来自对象槽位，也可来自小立即数常量	统一 PE 阵列，64 并行逐元素模式	依赖当前活动模上下文，便于支持对象乘小常量
基础模运算	pmac	psrc1, psrc2/imm, pdst	对对象执行逐元素模乘加，并将结果写回目的对象	统一 PE 阵列，64 并行逐元素模式	第二源既可来自对象槽位，也可来自小立即数常量

NTT / INTT	pntt	psrc -> pdst 或原地对象	执行一个 NTT stage，而非一次完整 NTT	统一 PE 阵列，128 并行蝶形/重排模式	单条指令内部完成本 stage 的 butterfly 计算、twiddle 推进和必要内部重排；twiddle 静态参数来自当前模上下文
NTT / INTT	pintt	psrc -> pdst 或原地对象	执行一个 INTT stage，而非一次完整 INTT	统一 PE 阵列，128 并行蝶形/重排模式	与 pntt 对称；twiddle 静态参数来自当前模上下文
Shuffle 配置	pshcfg	psrc	从对象槽位中装载当前 Shuffle 配置	控制路径 / 状态寄存器路径	Shuffle 配置对象先通过 dload 装入，再由 pshcfg 激活
Shuffle 执行	pshuf	psrc -> pdst	执行一次独立的真实数据重排	统一 PE 阵列，128 并行蝶形/重排模式	与 pntt/pintt 物理复用同一条 Shuffle 数据通路
随机 / 采样	pseed	imm21	以 21 bit 立即数设置当前 PRNG 种子	控制路径 / PRNG 状态路径	用于在线生成 FHE 密文中的掩码部分，种子可公开
随机 / 采样	psample	pdst	触发随机采样，按流式方式生成伪随机掩码数据并写入目的对象	PRNG 路径 + 本地写回路径	主要用于减少掩码数据带宽，不承担安全噪声在线生成
模上下文	pmodld	psrc	从对象槽位中装载当前活动模上下文	控制路径 / 状态寄存器路径	模上下文对象先通过 dload 装入，再由 pmodld 激活；其中可包含 twiddle 静态参数
同步	psync	无显式数据源/目的	建立 HPU 内部命令流的阶段同步点	控制路径 / 同步屏障路径	当其位于队首时，等待此前所有 custom0 内部操作完成并对统一 SRAM 可见，然后向核侧发出中断

3.4 本版指令集范围说明

综上，本版纳入 HPU 的指令包括以下 14 条：

- **外部访存**：`dload`、`dstore`
- **基础模运算**：`padd`、`psub`、`pmul`、`pmac`

- **NTT / INTT stage**：`pntt`、`pintt`
- **Shuffle**：`pshcfg`、`pshuf`
- **随机 / 采样**：`pseed`、`psample`
- **模上下文**：`pmodld`
- **同步**：`psync`

其中：

- `dload / dstore` 负责建立和维护活动对象槽位；
- `padd / psub / pmul / pmac` 对应统一 PE 阵列在 **64 并行逐元素模式** 下的基础计算能力；
- `pntt`、`pintt` 和 `pshuf` 对应统一 PE 阵列在 **128 并行蝶形/重排模式** 下的执行能力；
- `pseed` 用于设置当前 PRNG 的公开初始化参数，以支撑掩码在线生成；
- `pmodld` 用于从对象槽位装载当前活动模上下文；
- `psync` 用于建立 HPU 内部操作收敛后的显式同步边界。

4. HPU 指令编码表设计

基于前述指令助记符设计，本节对 HPU 指令的编码进行细化。本版同时覆盖两条路径：

- `custom0`：HPU 内部计算、重排、配置与同步指令
- `custom1`：外部访存指令 `dload / dstore`

与原方案相比，本版的主要变化包括：

1. 不再采用 `CLASS + OP` 的两级稀疏编码，而对 `custom0` 采用 **4 bit 扁平主操作码** `OPC`；
2. 多项式对象长度统一用不定式 `l` 表示；
3. 对象槽位只保留最小必要状态；
4. `dload / dstore` 被纳入统一编码设计，用于建立和维护对象槽位。

4.1 对象槽位的最小必要状态

本版中，每个活动对象槽位只维护以下最小必要状态：

- `V`：对象是否有效
- `BASE`：对象在本地 SRAM 中的起始位置
- `l`：对象的逻辑长度
- `FULL`：对象是否表示完整多项式
  - `1`：完整多项式
  - `0`：片段对象

其中：

- `V / BASE / l / FULL` 构成对象槽位的**架构可见最小状态**；
- `busy / dma_pending / dirty` 等运行时状态可作为微架构内部状态维护，但不纳入这里的最小架构状态集合。

4.2 custom0 编码总体约定

对于 `custom0`，由于 `opcode[6:0]` 固定为 `0001011`，因此 HPU 实际可用于内部编码的字段为 `inst[31:7]` 共 25 bit。
本版采用如下统一约定：

1. `inst[31:28]` 定义为 **4 bit 主操作码** `OPC`；
2. 对象槽位编号仍采用 **3 bit**；
3. `pntt / pintt` 使用 **4 bit stage 编号**；
4. `pseed` 仍放在配置类中，但其 `inst[27:7]` 的 **21 bit 全部解释为立即数种子** `IMM21`；
5. `pmul / pmac` 支持对象源和小立即数常量源两种模式。

4.3 custom0 基础格式

本版 `custom0` 采用以下四类基础格式：

- **AR3 格式**：三操作数基础算术类
- **STG 格式**：stage / transform 执行类
- **CFG 格式**：配置 / 控制类
- **SYNC 格式**：同步屏障类

4.3.1 AR3 格式：三对象基础算术类

该格式用于基础逐元素模运算类指令。

位段定义

位段	字段名	位宽	说明
inst[31:28]	OPC	4	主操作码
inst[27:25]	pdst	3	目的对象槽位
inst[24:22]	psrc1	3	源对象槽位 1
inst[21:14]	OP2	8	第二源对象槽位或小立即数常量
inst[13:10]	MODE	4	运算模式
inst[9:7]	FLAG	3	附加控制位

适用范围

该格式用于：

- `padd`
- `psub`
- `pmul`
- `pmac`

说明

- 当 `MODE[0] = 0` 时，`OP2[2:0]` 解释为 `psrc2`
- 当 `MODE[0] = 1` 时，`OP2[7:0]` 解释为 `cimm8`，即小立即数常量

4.3.2 STG 格式：stage / transform 执行类

该格式用于具有明显阶段性、变换性或流式控制特征的指令。

位段定义

位段	字段名	位宽	说明
inst[31:28]	OPC	4	主操作码
inst[27:25]	pdst	3	目的对象描述符
inst[24:22]	psrc	3	源对象描述符
inst[21:18]	IDX0	4	stage / 主模式索引
inst[17:14]	IDX1	4	次模式 / 参数索引
inst[13:10]	MODE	4	执行模式
inst[9:7]	FLAG	3	附加控制位

适用范围

该格式用于：

- `pntt`
- `pintt`
- `pshuf`
- `psample`

说明

`pntt / pintt`

- `pdst`：目的对象槽位
- `psrc`：源对象槽位

- `IDX0`： `stage_id`
- `IDX1`： stage 相关参数 / 布局模式
- `MODE / FLAG`： 执行模式与附加控制

其中， `psrc / pdst` 应指向完整多项式对象。

`pshuf`

- `pdst`： 目的对象槽位
- `psrc`： 源对象槽位
- `IDX0`： Shuffle 主模式
- `IDX1`： 参数化模式索引
- `MODE / FLAG`： 附加控制

`pshuf` 可作用于完整对象或片段对象。

`psample`

- `pdst`： 目的对象槽位
- `psrc`： 首版中未使用，置零
- `IDX0`： 采样模式 / 分布选择
- `IDX1`： 参数索引
- `MODE / FLAG`： 附加控制

`psample` 使用当前 `pseed` 所设置的 PRNG 状态生成掩码数据，并将其写入目的对象。

4.3.3 CFG 格式：配置 / 控制类

该格式用于更新当前活动状态，而不是直接完成大规模数据运算。

位段定义

位段	字段名	位宽	说明
inst[31:28]	OPC	4	主操作码
inst[27:25]	IDX0	3	槽位 / 主索引
inst[24:22]	IDX1	3	次索引 / 模式选择
inst[21:7]	CFG	15	配置字段

适用范围

该格式用于：

- `pshcfg`

- `pseed`
- `pmodld`

说明

`pshcfg`

- `IDX0`：源对象槽位
- `IDX1`：附加模式
- `CFG[14:0]`：保留或扩展控制字段

`pshcfg` 的语义是：从 `IDX0` 所指向的对象槽位中读取 Shuffle 配置对象，并装载到当前活动 Shuffle 配置状态中。

`pmodld`

- `IDX0`：源对象槽位
- `IDX1`：附加模式
- `CFG[14:0]`：保留或控制字段

`pmodld` 的语义是：从 `IDX0` 所指向的对象槽位中读取模上下文，并装载到当前活动模状态寄存器中。该模上下文中可以包含 NTT 所需的 twiddle 静态参数。

`pseed`

对 `pseed` 而言，CFG 格式中的字段解释与其他配置指令不同：
`inst[27:7]` 的 21 bit 全部解释为立即数种子 `IMM21`，而不再分解为 `IDX0 / IDX1 / CFG`。
这样无需为 `pseed` 再单独拆一类格式。

4.3.4 SYNC 格式：同步屏障类

该格式用于显式同步屏障指令。

位段定义

位段	字段名	位宽	说明
inst[31:28]	OPC	4	主操作码
inst[27:23]	TAG	5	同步标记
inst[22:20]	MODE	3	同步模式 / 预留
inst[19:7]	RSV	13	保留

适用范围

该格式目前仅用于：

- `psync`

说明

- `TAG` 可作为软件可见同步点标识
- `MODE` 当前可保留，用于未来扩展同步种类或中断策略
- `psync` 不携带源/目的对象，不参与普通计算

4.4 custom0 主操作码分配表

由于本版改为 4 bit 扁平主操作码，因此不再单独设置 `CLASS` 与类内 `OP` 两级字段，而直接按如下方式分配：

OPC[3:0]	指令	格式	说明
0000	padd	AR3	逐元素模加
0001	psub	AR3	逐元素模减
0010	pmul	AR3	逐元素模乘
0011	pmac	AR3	逐元素模乘加
0100	pntt	STG	单个 NTT stage
0101	pintt	STG	单个 INTT stage
0110	pshcfg	CFG	从对象槽位装载当前 Shuffle 配置
0111	pshuf	STG	执行独立真实重排
1000	pseed	CFG	以 21 bit 立即数设置 PRNG 种子
1001	psample	STG	执行随机采样
1010	pmodld	CFG	从对象槽位装载当前模上下文
1011	psync	SYNC	建立同步屏障并触发同步中断
1100	保留	-	预留扩展
1101	保留	-	预留扩展
1110	保留	-	预留扩展
1111	保留	-	预留扩展

4.5 custom1 外部访存指令编码

`dload / dstore` 采用 `custom1` 操作码，并继续沿用 RISC-V 风格字段组织。其编码如下：

位段	字段名	位宽	说明
inst[31:25]	funct7	7	首版保留
inst[24:20]	rs2	5	存放逻辑长度 l 的 GPR 编号
inst[19:15]	rs1	5	存放主存基址的 GPR 编号
inst[14]	DIR	1	0=dload，1=dstore
inst[13:12]	TYPE	2	dload 时解释为 LOAD_TYPE；dstore 时另作解释
inst[11:9]	obj_id	3	对象槽位编号
inst[8:7]	RSV2	2	保留
inst[6:0]	opcode	7	固定为 custom1

语义说明

当 `DIR = 0`（`dload`）时

`TYPE[1:0]` 解释为 `LOAD_TYPE`：

- `00`：普通片段对象
- `01`：完整多项式对象
- `10`：模上下文对象
- `11`：Shuffle 配置对象

此时：

- `rs1` 给出主存基址
- `rs2` 给出逻辑长度 `l`
- `obj_id` 给出目标对象槽位

HPU 在本地 SRAM 中为该对象分配 `BASE`，并在传输完成后置 `V=1`。
其中：

- `LOAD_TYPE=01` 表示该对象槽位应被解释为完整多项式对象；
- `LOAD_TYPE=10` 表示该对象槽位中存放的是模上下文；
- `LOAD_TYPE=11` 表示该对象槽位中存放的是 Shuffle 配置对象。

当 `DIR = 1`（`dstore`）时

- `TYPE[1]` 解释为 `REL`
  - `0`：导出后对象槽位保留

- `1`：导出后对象槽位释放
- `TYPE[0]`：首版保留

此时：

- `obj_id` 指向需要导出的对象槽位
- `rs1` 给出主存目标地址
- `rs2` 给出逻辑长度 `l`

4.6 本版编码方案的设计理由

综上，本版第 4 章相较上一版有三项关键调整：

1. 扁平化主操作码

不再使用稀疏的 `CLASS + OP` 两级结构，而改用 4 bit 主操作码。
这样节省出的位宽优先用于模式、片段控制和同步标记，而不是继续扩大操作码树。

2. 对象描述符仍保持 3 bit

对象描述符不再被解释为“很多完整多项式对象编号”，而被解释为**当前活动对象/片段视图槽位**。
因此，保留 3 bit 是合理的；继续扩宽并不能直接带来现实收益。

3. 明确区分“完整对象”与“片段对象”

- `pntt / pintt` 面向完整多项式对象
- `padd / psub / pmul / pmac / pshuf / psample` 可面向片段对象

4. `pseed` 保留在配置类中，但 `inst[27:7]` 整段解释为 `IMM21`；
5. `pmodld` 改为从对象槽位中装载模上下文，且该模上下文中可包含 twiddle 静态参数；
6. `pshcfg` 改为从对象槽位中装载 Shuffle 配置；
7. `dload / dstore` 被纳入统一编码体系，其中 `dload` 可区分普通片段对象、完整多项式对象、模上下文对象和 Shuffle 配置对象。

这样编码语义更贴近当前硬件存储现实，也更适合统一 SRAM + 流式执行的架构。

5. DMA带宽计算

5.1 分析条件与约束

这里考虑的对象是单次 **RelinOnly**，参数与约束如下：

$N = 65536, l = 3, DNUM = 1, \alpha = 3$

- 加速器工作频率为 **1 GHz**
- CPU 上单次 RelinOnly 的运行时间为 **48.113 ms**
- 目标加速比至少希望达到 **20×**

- 并行度固定为 $P = 64$
- 每个模数位宽为 **32 bit**，因此每个系数占 **4 Byte**
- 一个完整多项式的数据大小为
- $S_{\text{poly}} = 65536 \times 4 = 262144\ \text{B} = 256\ \text{KB}$
- **MA / MM / NTT 复用同一套算子**
- **MM** 计入流水延迟
- **NTT** 的重排按**流水化置换网络**建模
- 输出写回先不计入，只统计输入和中间结果造成的带宽压力

20× 加速对应的时间预算为：

$$T_{\text{budget}} = \frac{48.113}{20} = 2.40565\ ms$$

也就是单次 RelinOnly 必须在 **2.40565 ms** 内完成，才能达到 20×。

5.2 底层算子的 cycle 模型

由于三类算子复用同一套硬件，统一并行度记为 P 。

5.2.1 模加 MA

MA 为 1 级流水，因此一次多项式模加的时间为：

$$C_{\text{MA-poly}} = \frac{65536}{P}$$

在 $P = 64$ 时：

$$C_{\text{MA-poly}} = \frac{65536}{64} = 1024$$

也就是一次多项式级模加需要 **1024 cycles**。

5.2.2 模乘 MM

MM 为 5 级流水，计入首尾延迟后：

$$C_{\text{MM-poly}} = \frac{65536}{P} + 4$$

在 $P = 64$ 时：

$$C_{\text{MM-poly}} = 1024 + 4 = 1028$$

也就是一次多项式级模乘需要 **1028 cycles**。

5.2.3 NTT / INTT

NTT 单元每 cycle 可处理 $2P$ 个元素，因此在 $P = 64$ 时，每 cycle 处理 **128 个元素**。

对一个 $N = 65536$ 点多项式，$\log_2 N = 16$ 。单轮 NTT 包括两部分：

- 计算阶段：$\frac{65536}{2P}+6=\frac{32768}{P}+6$
- 重排阶段（流水化置换网络）：

$$\frac{65536}{2P}+\log_2 P-1=\frac{32768}{P}+\log_2 P-1$$

因此单轮时间为：

$$\frac{65536}{P}+\log_2 P+5$$

整次 NTT / INTT 的时间为：

$$C_{\text{NTT-poly}}=16\left(\frac{65536}{P}+\log_2 P+5\right)$$

代入 $P=64$ 和 $\log_2 64=6$：

$$C_{\text{NTT-poly}}=16(1024+6+5)=16\times 1035=16560$$

所以一次多项式级 NTT 或 INTT 需要 **16560 cycles**。

5.3 RelinOnly 到底层算子的映射

RelinOnly 可以拆成以下步骤：

- 对输入的多项式做 INTT
- 组内组合，把 l 个多项式扩展成 $l+\alpha$ 个多项式
- 对这 $l+\alpha$ 个多项式做 NTT
- 输入 $2DNUM(l+\alpha)$ 个 key poly，做乘加归约
- 对中间结果做 INTT
- 去除 α 部分
- 对 $2l$ 个多项式做 NTT
- 最后做模减，得到输出

汇总后，整个 RelinOnly 对应的底层操作数为：

- **39 次多项式级 MM**
- **24 次多项式级 MA**
- **27 次多项式级 NTT/INTT**

因此总 cycle 数为：

$$C_{\text{Relin}}=39\times 1028+24\times 1024+27\times 16560=511788$$

也就是：

$$T_{\text{comp}}=511788\text{ cycles}=0.511788\text{ ms}$$

所以在 $P = 64$ 下，如果完全不受通信影响，单次 RelinOnly 的纯计算加速比为：

$$\text{Speedup}_{\text{compute}} = \frac{48.113}{0.511788} \approx 94.0\times$$

这说明：在 $P = 64$ 时，系统已经远远不是“算力不够 20×”的问题，而是更可能受带宽组织方式影响。

5.4 两种缓存场景下的带宽模型

5.4.1 全缓存

在全缓存场景下，假设所有中间结果都留在片上，因此外部只需要输入：

- 初始输入：3 个多项式
- key：12 个多项式

总共：

$$3 + 12 = 15 \text{ 个多项式}$$

对应总输入数据量为：

$$15 \times 262144 = 3932160 \text{ B} = 3.75 \text{ MiB}$$

不过这里不能简单写成“总通信时间 < 总计算时间”。因为步骤 4 依赖 key，所以要区分两段：

- 前半段（步骤 1–3）计算时间：
- $T_{\text{pre}} = 0.164436 \text{ ms}$
- 后半段（步骤 4–8）剩余时间：
- $T_{\text{post}} = 0.347352 \text{ ms}$

如果带宽不足以在前半段把 12 个 key poly 全部搬完，那么步骤 4 会等待通信。

因此，全缓存下的总时间是分段的：

- 当带宽较低时：$T_{\text{total}} = 0.347352 \text{ ms} + \frac{3932160}{B}$
- 当带宽足够高，12 个 key 可以在前半段完全隐藏时：$T_{\text{total}} = 0.511788 \text{ ms} + \frac{786432}{B}$

其中 B 的单位是 Byte/s。

而要让步骤 4 完全不等待，带宽至少要达到：

$$B \geq \frac{3145728}{0.164436 \times 10^{-3}} \approx 19.13 \text{ GB/s}$$

也就是说：

- **1.91 GB/s** 左右已经足够达到 **20×**
- **19.13 GB/s** 左右可以让 key 基本完全隐藏，不再因步骤依赖额外等待

5.4.2 无缓存

在无缓存场景下，采用最简单的上界模型：

- 每次 **MM / MA** 都需要：
  - 读 2 个多项式
  - 写 1 个多项式
- 每次 **NTT / INTT** 都需要：
  - 读 1 个多项式
  - 写 1 个多项式

因此总外部传输多项式数为：

$$3(39+24)+2\times 27=243$$

对应总外部数据量为：

$$243\times 262144=63700992\ \mathrm{B}=60.75\mathrm{MB}$$

在这个模型下，由于几乎每一步都要靠 DMA 继续喂数据，通信与计算很难像全缓存那样只通过一个关键窗口隐藏。因此这里取一个更保守的对照模型：

$$T_{\text{total}}=T_{\text{comp}}+\frac{63700992}{B}$$

也就是：

$$T_{\text{total}}=0.511788\ \text{ms}+\frac{63700992}{B}$$

要求达到 20×，则需满足：

$$0.511788+\frac{63700992}{B}\le 2.40565$$

解得：

$$B\ge 33.64\ \text{GB/s}$$

因此，无缓存场景下，要达到 20×，带宽门槛大约是：**33.64 GB/s**

这比全缓存场景的 **1.91 GB/s** 明显高很多，也说明中间结果是否能留在片上，对带宽要求影响极大。

以下列出几种典型带宽下的加速比：